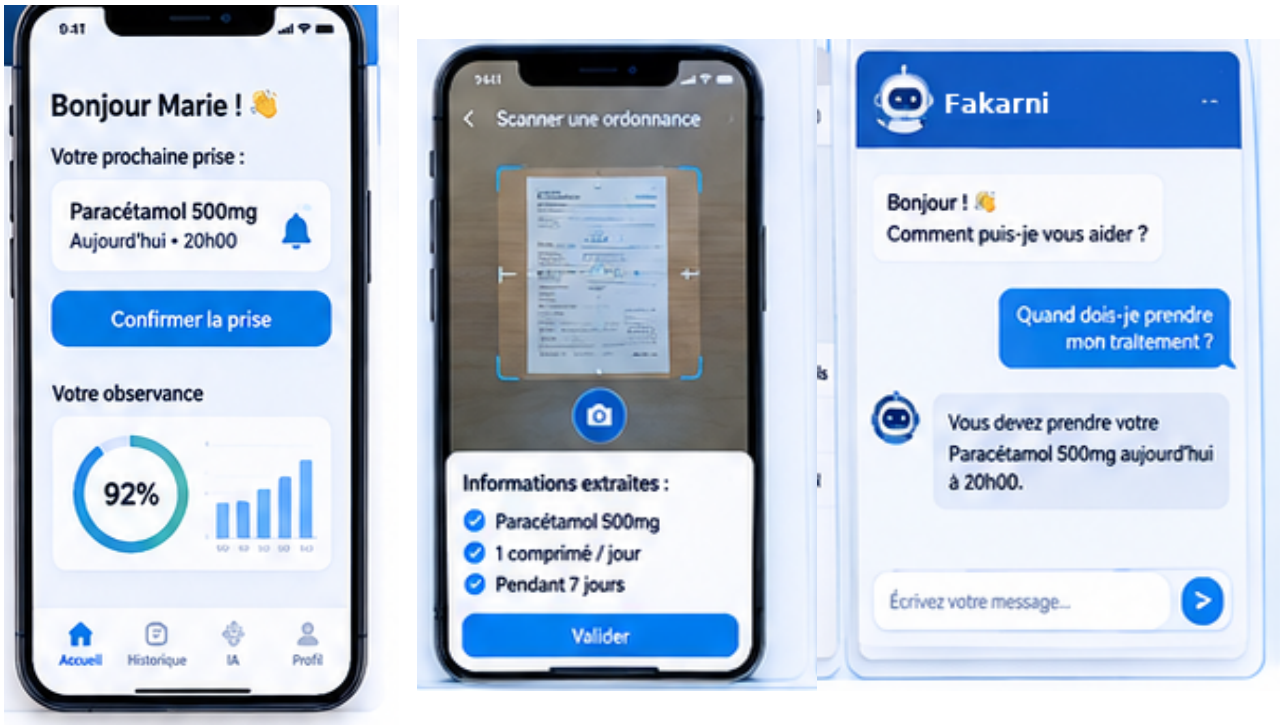
Fakarni

Assistant intelligent de suivi des médicaments

Moins d’oublis, un meilleur suivi au quotidien.

Fakarni signifie « rappelle-moi » en darija.



Type de solution : application Web et mobile avec modules d’intelligence artificielle
Méthodologie : SCRUM pour le développement — CRISP-DM pour la partie IA
Public cible : patients, aidants familiaux et professionnels de santé autorisés
Module : Projet de développement — **Équipe :** 4 étudiants

Sommaire

1. Résumé exécutif	12. Choix technologiques (stack complète)
2. Contexte et problématique	13. Architecture technique
3. Maquettes et interfaces envisagées	14. Modèle de données et API
4. État de l’art : étude comparative	15. Modules intelligents : conception détaillée
5. Acteurs du système	16. Partie IA : démarche CRISP-DM
6. Besoins fonctionnels	17. Qualité, tests et intégration continue
7. Besoins non fonctionnels	18. Outils de collaboration
8. Diagramme de cas d’utilisation global	19. Sécurité, confidentialité et limites médicales
9. Product Backlog estimé	20. Risques et mesures de réduction
10. Méthodologie SCRUM et répartition des rôles	21. Conclusion et perspectives
11. Planification des sprints et releases	22. Questions probables du jury et réponses

Ce qui a été renforcé dans cette version

- Diagramme de cas d’utilisation rendu graphiquement (et non plus en code PlantUML).
- État de l’art avec une grille de critères comparative et sourcée.
- Product Backlog estimé en story points (suite de Fibonacci) avec vélocité par sprint.
- Stack complétée : Celery + Redis (tâches planifiées), FastAPI (service IA isolé), JWT, WeasyPrint, GitHub Actions.
- Pipeline OCR détaillé avec étape d’extraction structurée des champs (regex + spaCy / LLM).
- Assistant RAG précisé : modèle d’embeddings, base vectorielle Chroma, LLM, garde-fous.
- Modèle de prédiction précisé : variables, cible, baseline, générateur de données synthétiques, métriques.
- Nouveaux chapitres : architecture technique (schéma), modèle de données, endpoints API, tests et CI.
- Périmètre clarifié : Flutter pour le patient (mobile), React pour le tableau de bord multi-rôles (web).

1. Résumé exécutif

Fakarni est une application Web et mobile destinée à aider les personnes à organiser et suivre leurs traitements médicamenteux. Elle centralise les informations saisies par l'utilisateur, planifie les prises, envoie des rappels, enregistre les confirmations et présente un historique de suivi. Trois modules intelligents — lecture d'ordonnance par OCR, prédiction du risque d'oubli et assistant conversationnel RAG — facilitent la saisie, l'analyse des habitudes et l'utilisation de l'application.

Le projet est conçu comme un outil d'organisation et de suivi. Il ne prescrit pas de médicaments, ne modifie pas les doses et ne remplace pas l'avis d'un médecin ou d'un pharmacien.

Objectifs principaux

- Réduire les oublis en aidant l'utilisateur à suivre un planning clair.
- Faciliter l'enregistrement des médicaments et des horaires de prise.
- Permettre la lecture assistée d'une ordonnance par OCR, avec validation humaine obligatoire.
- Présenter des statistiques compréhensibles sur les prises confirmées et non confirmées.
- Permettre un partage limité et consenti des informations avec un aidant ou un professionnel autorisé.

2. Contexte et problématique

Les personnes qui suivent plusieurs traitements peuvent rencontrer des difficultés d'organisation : horaires multiples, changements de routine, oublis, difficulté à relire une ordonnance ou à retrouver l'historique des prises. Les rappels classiques peuvent aider, mais ils ne fournissent pas toujours un suivi centralisé ni une analyse des habitudes.

Problématique : comment concevoir une application simple et sécurisée qui accompagne les utilisateurs dans l'organisation de leurs traitements, les aide à limiter les oublis et facilite le suivi grâce à des fonctionnalités intelligentes ?

Solution proposée

- Un espace personnel pour gérer les médicaments et les traitements.
- Un calendrier des prises avec notifications configurables.
- Une action simple pour confirmer, reporter ou signaler une prise non effectuée.
- Un historique et des indicateurs d'observance calculés à partir des événements enregistrés.
- Un scan d'ordonnance qui extrait le texte et propose un formulaire prérempli à vérifier.
- Un indicateur expérimental de risque d'oubli par créneau horaire.
- Un assistant conversationnel limité à l'aide à l'utilisation et aux données autorisées de l'application.

3. Maquettes et interfaces envisagées

Les écrans ci-dessous sont des maquettes conceptuelles illustrant la direction graphique du produit. Ce ne sont pas des captures d'une application déjà développée ; les libellés et chiffres visibles sont fictifs. Ils seront remplacés par les écrans Figma de l'équipe au fil des sprints.



Mobile — accueil, prochain rappel et observance



Mobile — scan d'ordonnance (OCR)



Web — tableau de bord, statistiques et risque d'oubli



Assistant conversationnel IA

4. État de l’art : étude comparative

L’étude de l’état de l’art sert à comprendre les solutions existantes et à positionner Fakarni. Les informations ci-dessous proviennent des fiches officielles des stores et des sites des éditeurs (consultés en novembre 2026) ; les offres peuvent varier selon le pays et la version.

Solution	Positionnement	Plateformes	Modèle économique	Points forts
Medisafe	Gestion des médicaments et rappels	iOS, Android	Freemium ; depuis 2026, la version gratuite est limitée à 2 médicaments	Rappels persistants, Medfriend (alerte d’un proche en cas d’oubli), vérification d’interactions, rappels de renouvellement
MyTherapy	Rappels + journal de santé	iOS, Android	Gratuit (options payantes sans publicité)	Journal de symptômes et d’humeur, 20+ mesures de santé, rapport PDF partageable avec le médecin
CareClinic	Suivi global de santé	iOS, Android, Web	Freemium (abonnement Premium)	Trackers multiples (symptômes, nutrition, sommeil), plans de soins, synchronisation avec l’application Web
Fakarni (proposé)	Suivi médicamenteux assisté par IA	Android, iOS (Flutter) + Web (React)	Projet académique (gratuit)	OCR d’ordonnance avec validation, prédiction du risque d’oubli, assistant RAG, partage révocable

Grille comparative par critères

Critère	Medisafe	MyTherapy	CareClinic	Fakarni
Rappels de prise configurables	**Oui**	**Oui**	**Oui**	**Oui**
Historique / journal des prises	**Oui**	**Oui**	**Oui**	**Oui**
Application Web	Non	Non	**Oui**	**Oui**
Partage avec un aidant	**Oui**	Partiel (rapport PDF)	**Oui**	**Oui** (révocable, granulaire)
Rapport exportable	**Oui**	**Oui**	**Oui**	**Oui**
Scan d’ordonnance par OCR	Non mis en avant	Non mis en avant	Non mis en avant	**Oui** (validation obligatoire)
Prédiction du risque d’oubli (ML)	Non mis en avant	Non mis en avant	Partiel (analyses Premium)	**Oui** (expérimental, explicable)
Assistant conversationnel IA	Non mis en avant	Non mis en avant	Non mis en avant	**Oui** (RAG limité à l’app)
Entièrement gratuit	Non	Partiel	Non	**Oui**

Positionnement : les concurrents couvrent bien les rappels et le journal de prise. La différenciation de Fakarni réside dans l’intégration cohérente, dans un même parcours, de la saisie assistée par OCR, du suivi, de l’analyse prédictive des habitudes et du partage sous contrôle du patient. « Non mis en avant » signifie que la fonctionnalité n’apparaît pas dans la présentation officielle de l’application, ce qui ne prouve pas son absence.

Limites à prendre en compte

- L’OCR peut mal lire un nom, un dosage ou une fréquence : toute extraction doit être vérifiée.
- Un historique incomplet ne permet pas de conclure qu’un médicament n’a pas été pris.
- Un modèle prédictif peut se tromper et doit être présenté comme une aide non médicale.

- Les notifications dépendent des permissions du téléphone, de la connexion et des réglages du système.

5. Acteurs du système

Acteur	Rôle et principales actions
Patient (principal)	Créer son compte, gérer son profil, saisir ses médicaments, planifier les prises, confirmer les événements, consulter l'historique et les statistiques, scanner une ordonnance, utiliser l'assistant, gérer les partages.
Aidant familial	Accéder uniquement aux informations partagées par le patient ; consulter les alertes et l'historique autorisés.
Professionnel de santé	Consulter les rapports explicitement partagés par le patient et accompagner le suivi. Aucun accès automatique.
Administrateur	Gérer les comptes, les rôles et les paramètres techniques ; superviser le service sans consulter inutilement les données de santé.
Système (acteur secondaire)	Planificateur Celery et Firebase Cloud Messaging : déclenchent l'envoi des rappels à l'heure prévue.

6. Besoins fonctionnels

ID	Besoin	Description
BF01	Inscription et connexion	Créer un compte, s'authentifier (JWT) et se déconnecter.
BF02	Gestion du profil	Modifier les informations, le fuseau horaire et les préférences de notification.
BF03	Gestion des médicaments	Ajouter, modifier, archiver et consulter les médicaments.
BF04	Planification	Définir les horaires, la fréquence et la période du traitement.
BF05	Notifications	Recevoir des rappels push configurables, avec relance si non confirmés.
BF06	Suivi des prises	Confirmer, reporter ou déclarer une prise non effectuée.
BF07	Historique	Consulter les événements passés et leur statut.
BF08	OCR ordonnance	Importer une photo/PDF, extraire le texte et préremplir une fiche à vérifier.
BF09	Analyse des habitudes	Calculer l'observance et prédire les créneaux à risque d'oubli.
BF10	Assistant conversationnel	Répondre aux questions sur l'application et les données accessibles de l'utilisateur.
BF11	Tableau de bord	Afficher les prochains rappels et les indicateurs de suivi.
BF12	Partage contrôlé	Autoriser et révoquer l'accès d'un aidant ou d'un professionnel.
BF13	Rapport	Exporter un résumé des prises en PDF.
BF14	Administration	Gérer les comptes, rôles et paramètres de la plateforme.

7. Besoins non fonctionnels

ID	Exigence	Critère mesurable
BNF01	Sécurité	JWT (access 15 min / refresh), mots de passe hachés (Argon2/PBKDF2), RBAC, HTTPS, chiffrement des champs sensibles.
BNF02	Confidentialité	Consentement explicite, minimisation des données, accès révocable, conformité RGPD.
BNF03	Performance	Temps de réponse API < 500 ms (p95) pour les écrans courants ; OCR < 10 s par page.

ID	Exigence	Critère mesurable
BNF04	Disponibilité	Rappel envoyé avec un retard < 1 min ; reprise automatique (retry Celery) en cas d'échec.
BNF05	Ergonomie	Confirmation d'une prise en 2 actions maximum ; contrastes WCAG AA ; police adaptable.
BNF06	Compatibilité	Android 8+, iOS 14+, dernières versions de Chrome, Firefox, Edge et Safari.
BNF07	Fiabilité	Stockage en UTC, conversion selon le fuseau de l'utilisateur ; journalisation des événements.
BNF08	Évolutivité	Service IA découplé (FastAPI) ; conteneurs Docker indépendants.
BNF09	Maintenabilité	Couverture de tests backend ≥ 70 % ; linting (ruff, eslint, dart analyze) ; documentation OpenAPI.
BNF10	Traçabilité	Journal d'audit des accès et des actions sensibles (partage, export, consultation par un tiers).

8. Diagramme de cas d’utilisation global

Le diagramme résume les interactions majeures. Les autorisations de l’aidant et du professionnel sont conditionnées par le consentement du patient. Les cas d’utilisation intégrant un module intelligent sont mis en évidence en vert.

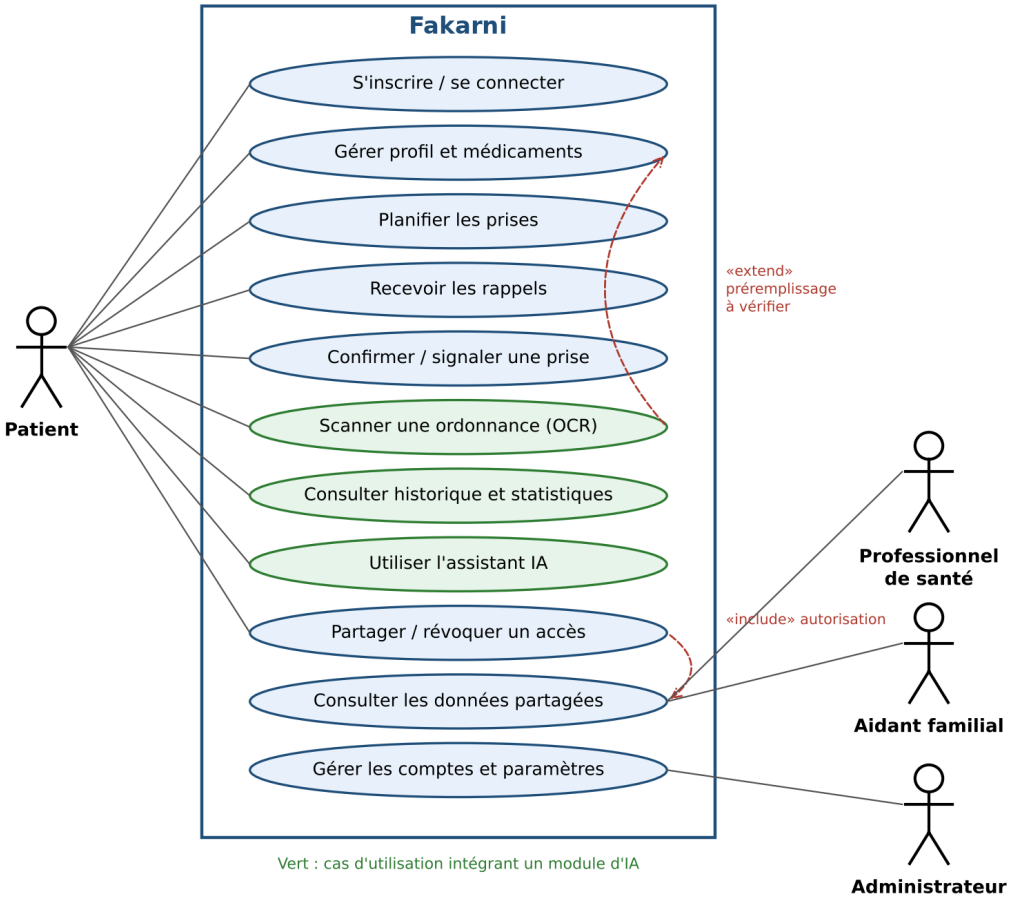


Figure 1 — Diagramme de cas d’utilisation global de Fakarni

9. Product Backlog estimé

Les User Stories sont priorisées avec MoSCoW (Must, Should, Could) et estimées en story points (suite de Fibonacci : 1, 2, 3, 5, 8, 13) par Planning Poker. Les estimations seront affinées lors de chaque Sprint Planning.

ID	User Story	Priorité	Points	Sprint
US01	En tant que patient, je veux créer un compte pour retrouver mon espace personnel.	Must	3	S1
US02	En tant qu'utilisateur, je veux me connecter de manière sécurisée (JWT).	Must	3	S1
US03	En tant que patient, je veux gérer mon profil, mon fuseau horaire et mes préférences.	Must	2	S1
US17	En tant qu'administrateur, je veux gérer les comptes et les rôles.	Must	5	S1
US04	En tant que patient, je veux ajouter, modifier et archiver un médicament.	Must	5	S2
US05	En tant que patient, je veux planifier les horaires de prise.	Must	5	S2
US06	En tant que patient, je veux recevoir un rappel à l'heure programmée.	Must	8	S2
US07	En tant que patient, je veux confirmer, reporter ou signaler une prise.	Must	3	S2
US08	En tant que patient, je veux consulter mon historique de prises.	Must	3	S2
US09	En tant que patient, je veux scanner une ordonnance pour préremplir une fiche.	Should	13	S3
US10	En tant que patient, je veux vérifier et corriger les données extraites avant enregistrement.	Must	5	S3
US11	En tant que patient, je veux consulter des statistiques sur mes prises.	Should	5	S3
US12	En tant que patient, je veux connaître les créneaux où je risque d'oublier une prise.	Should	8	S3
US13	En tant que patient, je veux poser des questions sur l'application à un assistant.	Should	8	S4
US14	En tant que patient, je veux partager certaines données avec un aidant.	Could	5	S4
US15	En tant que patient, je veux révoquer un accès partagé.	Must	3	S4
US16	En tant que patient, je veux exporter un rapport PDF de mon suivi.	Could	3	S4

Sprint	User Stories	Total points
Sprint 1	US01, US02, US03, US17	13
Sprint 2	US04 à US08	24
Sprint 3	US09 à US12	31
Sprint 4	US13 à US16 + stabilisation	19 (+ tests)
Total	17 User Stories	**87**

Note : le Sprint 3, le plus chargé, concentre deux modules d'IA ; l'assistant RAG a donc été déplacé au Sprint 4 pour équilibrer la charge (environ 20 à 30 points par sprint pour une équipe de 4).

Critères d'acceptation — exemples

- **US06 — Rappel :** le patient reçoit une notification à l'horaire défini (± 1 min) si les permissions sont activées ; le rappel est associé au bon médicament et au bon créneau ; l'événement est journalisé ; une relance est envoyée après 15 min sans confirmation.
- **US09/US10 — OCR :** l'utilisateur importe une image ; le système affiche le texte extrait et les champs proposés avec un indicateur de confiance ; aucun traitement n'est enregistré sans confirmation explicite.

- **US12 — Risque d'oubli :** l'indicateur n'apparaît qu'après 14 jours d'historique ; il est accompagné d'une explication (ex. « vos prises du soir sont souvent non confirmées ») et peut être désactivé.
- **US15 — Révocation :** après révocation, toute requête de l'aidant sur les données du patient renvoie une erreur 403 ; l'action est inscrite au journal d'audit.

10. Méthodologie SCRUM et répartition des rôles

SCRUM est retenu pour livrer progressivement un produit testable, recueillir les retours et adapter les priorités. Le projet est organisé en quatre sprints de deux semaines.

Événement / rôle	Application au projet
Product Owner	Porte la vision, priorise le Product Backlog, rédige et valide les critères d'acceptation.
Scrum Master	Facilite les événements SCRUM, lève les obstacles, tient le sprint board Jira à jour.
Developers	Conçoivent, développent, testent et intègrent les fonctionnalités à chaque sprint.
Sprint Planning	Sélection des User Stories, estimation et définition de l'objectif du sprint.
Daily Scrum	Point de 15 min (Discord) sur l'avancement et les obstacles.
Sprint Review	Démonstration de l'incrément et collecte des retours de l'encadrant.
Sprint Retrospective	Identification des améliorations pour le sprint suivant.

Répartition des rôles dans l'équipe

Membre	Rôle SCRUM	Responsabilité technique principale
[Nom 1]	Product Owner + Developer	Backlog, critères d'acceptation, application Web React (tableau de bord, partage, administration).
[Nom 2]	Scrum Master + Developer	Organisation SCRUM, CI/CD GitHub Actions, Docker, application mobile Flutter.
[Nom 3]	Developer — Backend	API Django REST, modèle de données PostgreSQL, JWT/RBAC, Celery + Redis, notifications FCM, rapport PDF.
[Nom 4]	Developer — IA / Data	Service FastAPI : pipeline OCR, modèle de prédiction (CRISP-DM), assistant RAG, évaluation.

Dans SCRUM, les Developers forment une seule équipe responsable de l'incrément : ces responsabilités servent à répartir le travail, sans créer de silos. Les revues de code croisées sont obligatoires.

11. Planification des sprints et releases

Période	Sprint / release	Livrables techniques
Semaines 1–2	Sprint 1 — Fondations (Release 0.1 : socle)	Maquettes Figma validées ; dépôts GitHub et pipeline CI ; docker-compose (Django, PostgreSQL, Redis) ; modèle de données ; authentification JWT ; profils et rôles.
Semaines 3–4	Sprint 2 — Suivi des traitements (Release 0.2 : MVP utilisable)	CRUD médicaments et plannings ; tâches Celery Beat ; notifications FCM ; confirmation des prises ; historique ; écrans Flutter patient.
Semaines 5–6	Sprint 3 — Intelligence (Release 0.3 : IA)	Service FastAPI ; pipeline OCR + écran de vérification ; statistiques ; générateur de données synthétiques ; modèle de prédiction et endpoint /predict.
Semaines 7–8	Sprint 4 — Finalisation (Release 1.0 : candidate)	Assistant RAG ; partage contrôlé et révocation ; rapport PDF ; tests de bout en bout ; durcissement de la sécurité ; documentation et démonstration.

Definition of Done (DoD)

- Le code est intégré dans la branche *main* via une pull request relue par au moins un membre.
- Le pipeline CI passe (lint + tests unitaires et d'intégration).
- Les critères d'acceptation de la User Story sont satisfaits.
- Les endpoints sont documentés (OpenAPI / Swagger) et testés dans Postman.
- La fonctionnalité est démontrable dans l'environnement Docker de test.

12. Choix technologiques (stack complète)

Renforcement technique — ajout de Celery + Redis, FastAPI, JWT, spaCy, LangChain/Chroma, WeasyPrint et GitHub Actions ; précision du LLM et du modèle d'embeddings.

Couche	Technologie	Justification
Mobile (patient)	Flutter (Dart)	Une seule base de code Android/iOS ; notifications locales et push ; caméra pour le scan.
Web (multi-rôles)	React.js + Vite, React Query, Recharts	Tableau de bord patient, aidant, professionnel et administrateur ; graphiques d'observance.
Backend / API	Python, Django REST Framework	ORM, administration intégrée, sérialisation et permissions robustes.
Authentification	djangorestframework-simplejwt	Jetons JWT access/refresh adaptés aux clients mobile et Web ; permissions par rôle.
Tâches planifiées	Celery + Celery Beat, Redis	Planification fiable des rappels, relances, génération asynchrone des rapports et des appels IA.
Base de données	PostgreSQL	Base relationnelle robuste ; contraintes d'intégrité ; champ JSONB pour les résultats OCR.
Service IA	FastAPI (Python)	Isole les dépendances lourdes (OpenCV, modèles) ; documentation automatique ; asynchrone.
OCR	OpenCV + Tesseract (langue fra) via pytesseract	Prétraitement d'image puis extraction de texte ; open source et exécutable en local.
Extraction des champs	Regex + spaCy (fr_core_news_md) ; option LLM en sortie JSON	Transforme le texte brut en champs structurés : nom, dosage, fréquence, durée.
Machine Learning	Pandas, scikit-learn, joblib	Régression logistique et arbre de décision, explicables ; sérialisation du modèle.
Assistant IA	LangChain + Chroma ; embeddings paraphrase-multilingual-MiniLM ; LLM Mistral via API ou Ollama (local)	RAG limité à la documentation de l'application et aux données de l'utilisateur.
Notifications	Firebase Cloud Messaging	Envoi de push vers Android et iOS.
Rapports	WeasyPrint	Génération de PDF à partir de gabarits HTML Django.
Tests	pytest, pytest-django, Flutter test, Vitest, Postman	Tests unitaires, d'intégration et d'API.
DevOps	Docker, docker-compose, GitHub Actions	Environnement reproductible ; intégration continue à chaque pull request.

13. Architecture technique

Renforcement technique — le service IA est isolé dans un microservice FastAPI et les rappels passent par une file de tâches Celery/Redis.

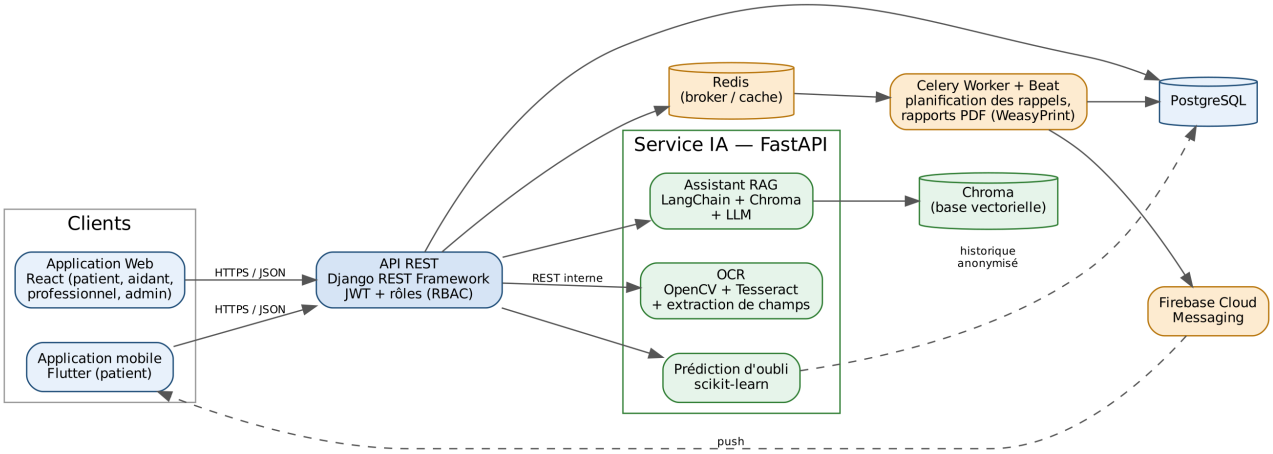


Figure 2 — Architecture technique de Fakarni

Composant	Responsabilité
Clients Flutter / React	Interfaces patient (mobile) et multi-rôles (Web) ; appels API sécurisés en HTTPS/JSON.
API REST Django	Authentification JWT, autorisations par rôle, logique métier, validation des données, journal d'audit.
Celery Worker + Beat	Toutes les minutes, sélectionne les prises à venir, envoie les rappels via FCM, relance après 15 min, génère les PDF.
Service IA FastAPI	Endpoints `/ocr`, `/predict`, `/chat` appelés uniquement par le backend (réseau Docker interne).
PostgreSQL / Redis / Chroma	Données métier ; file de messages et cache ; index vectoriel de l'assistant.

Flux principal : l'utilisateur importe une ordonnance → le service OCR propose des champs → l'utilisateur vérifie et corrige → le backend enregistre les informations validées → Celery Beat génère les rappels → l'utilisateur confirme les prises → le tableau de bord présente l'historique, les statistiques et le risque d'oubli.

14. Modèle de données et API

Renforcement technique — modèle de données principal et liste des endpoints, absents de la version précédente.

Entités principales

Entité	Attributs clés	Relations
User	id, email, mot de passe haché, rôle (PATIENT, CAREGIVER, PRO, ADMIN), fuseau horaire	1–N Medication, 1–N ShareGrant
Medication	id, nom, dosage, forme, instructions, statut (actif/archivé), source (manuel/OCR)	N–1 User, 1–N Schedule
Schedule	id, heure, jours, date début/fin, fréquence	N–1 Medication, 1–N IntakeEvent
IntakeEvent	id, prévu_à (UTC), statut (TAKEN, SNOOZED, MISSED, PENDING), confirmé_à, délai_réponse	N–1 Schedule

Entité	Attributs clés	Relations
PrescriptionScan	id, fichier, texte OCR, champs extraits (JSONB), confiance, validé (bool)	N–1 User
ShareGrant	id, patient, bénéficiaire, périmètre (historique, alertes, rapports), expire_le, révoqué_le	N–1 User (×2)
AuditLog	id, acteur, action, ressource, horodatage, IP	N–1 User

Endpoints principaux (extrait)

Méthode	Endpoint	Rôle
POST	/api/auth/register, /api/auth/token, /api/auth/refresh	Inscription, connexion, renouvellement du jeton
GET/POST/PATCH	/api/medications/, /api/medications/{id}/	Gestion des médicaments
GET/POST	/api/schedules/	Planification des prises
GET/PATCH	/api/intakes/?date=…, /api/intakes/{id}/	Historique ; confirmation / report / prise manquée
POST	/api/scans/ → /api/scans/{id}/validate/	Envoi d’ordonnance au service OCR, puis validation
GET	/api/insights/adherence/, /api/insights/risk/	Statistiques et risque d’oubli
POST	/api/assistant/chat/	Question à l’assistant RAG
POST/DELETE	/api/shares/, /api/shares/{id}/	Création et révocation d’un partage
GET	/api/reports/pdf/?from=…&to=…	Export du rapport PDF

15. Modules intelligents : conception détaillée

Renforcement technique — chaque module d'IA est décrit par son pipeline, ses technologies, ses entrées/sorties et ses garde-fous.

15.1 OCR d'ordonnance (vision par ordinateur + NLP)

Tesseract seul renvoie du texte brut et lit mal les documents photographiés. Le pipeline ajoute un prétraitement d'image et une étape d'extraction structurée afin de produire un formulaire réellement prérempli.

```
Image / PDF
  → [OpenCV] niveaux de gris, débruitage, binarisation adaptative, correction d'inclinaison
(deskew), recadrage
  → [Tesseract, lang=fra, psm 6] texte + score de confiance par mot
  → [Extraction] regex (dosages : « 500 mg », fréquences : « 2 fois par jour », durées : « pendant 7
jours »)
      + spaCy (entités) + correspondance approximative (RapidFuzz) avec une liste de médicaments de
référence
  → JSON {nom, dosage, fréquence, durée, confiance} → écran de vérification → validation par
l'utilisateur
```

- **Option avancée :** envoyer le texte OCR à un LLM avec un schéma JSON imposé pour mieux gérer les formulations variées ; le résultat reste une proposition.
- **Évaluation :** jeu de 30 à 50 ordonnances fictives annotées ; métriques CER (taux d'erreur par caractère) et exactitude par champ.
- **Limite assumée :** les ordonnances manuscrites sont hors périmètre du MVP ; les champs à faible confiance sont surlignés en orange.

15.2 Prédiction du risque d'oubli (Machine Learning supervisé)

Élément	Choix
Problème	Classification binaire : une prise planifiée sera-t-elle non confirmée ?
Cible	`missed` = 1 si le statut final est MISSED ou sans confirmation après 2 h, sinon 0.
Variables	Heure prévue, créneau (matin/midi/soir/nuit), jour de la semaine, week-end, nombre de prises du jour, taux d'oubli sur 7 jours, taux d'oubli du même créneau, délai moyen de réponse, ancienneté du traitement.
Baseline	Prédire l'oubli si le taux d'oubli historique du créneau dépasse 30 %.
Modèles	Régression logistique (coefficients interprétables) et arbre de décision (profondeur ≤ 5) ; sélection par validation croisée temporelle (TimeSeriesSplit).
Métriques	Rappel (priorité : ne pas rater les oublis), précision, F1, ROC-AUC, matrice de confusion.
Sortie	Score de risque par créneau + explication textuelle des variables dominantes ; seuil ajustable ; indicateur désactivable.

15.3 Assistant conversationnel (RAG)

```
Question utilisateur
  → [Garde-fou d'entrée] détection des questions médicales (dosage, arrêt, interactions) → réponse
de redirection vers un professionnel
  → [Récupération] Chroma : top-4 extraits de la documentation de l'application (FAQ, guide)
      + données de l'utilisateur via l'API (prochaines prises, observance), filtrées par user_id
  → [LLM] Mistral (API) ou modèle local via Ollama, avec un prompt système restrictif
  → Réponse + sources citées
```

- **Embeddings :** paraphrase-multilingual-MiniLM-L12-v2 (sentence-transformers), adapté au français ; découpage en blocs de 500 caractères avec chevauchement.
- **Sécurité :** le LLM n'accède jamais directement à la base ; il ne reçoit que des données déjà autorisées par l'API (pas de fuite entre utilisateurs).

- **Évaluation :** 40 questions de test (aide, données, questions médicales pièges) ; taux de bonnes réponses et taux de refus correct pour les questions médicales.

15.4 Rappels personnalisés (règles métier)

Ce n'est pas un module d'IA à part entière : à partir du score de risque (15.2), des règles proposent des ajustements — relance supplémentaire sur un créneau à risque, rappel 10 min plus tôt si le délai de réponse moyen est long. Toute suggestion est soumise à l'accord de l'utilisateur et ne modifie jamais l'horaire prescrit.

16. Partie IA : démarche CRISP-DM

Phase	Application à Fakarni
1. Compréhension métier	Objectif : repérer les créneaux où les prises sont souvent non confirmées afin d'améliorer les rappels. Critère de succès : rappel ≥ 0,70 et F1 supérieur à la baseline.
2. Compréhension des données	Événements de prise (prévu, confirmé, statut, délai). Analyse exploratoire : taux d'oubli par créneau et par jour, distribution des délais.
3. Préparation des données	Générateur synthétique (Python/NumPy) simulant 200 patients sur 90 jours avec des profils distincts (régulier, oublie le soir, oublie le week-end, irrégulier) ; nettoyage, encodage, création de variables, découpage temporel train/test.
4. Modélisation	Baseline par règle, puis régression logistique et arbre de décision ; réglage des hyperparamètres par GridSearchCV avec TimeSeriesSplit ; gestion du déséquilibre de classes (class_weight=balanced).
5. Évaluation	Précision, rappel, F1, ROC-AUC, matrice de confusion ; analyse des faux négatifs ; comparaison avec la baseline ; documentation des limites et biais du jeu synthétique.
6. Déploiement	Modèle sérialisé (joblib) chargé par FastAPI ; endpoint /predict ; réentraînement hebdomadaire par une tâche Celery ; suivi de la dérive des performances.

Données et éthique

- Commencer avec un jeu de données synthétique pour tester le pipeline et l'interface.
- Ne pas entraîner de modèle sur des données personnelles ou médicales sans base légale, consentement approprié et mesures de sécurité.
- Documenter la provenance des données, les limites, les biais possibles et les performances.
- Ne pas présenter le score de risque comme un diagnostic ou une probabilité médicale validée.

17. Qualité, tests et intégration continue

Renforcement technique — stratégie de tests et pipeline CI explicites.

Niveau	Outils	Ce qui est testé
Unitaires backend	pytest, pytest-django, factory_boy	Modèles, sérialiseurs, permissions par rôle, calcul de l'observance.
Tâches asynchrones	pytest + Celery en mode eager	Sélection des prises à rappeler, relance, gestion des fuseaux horaires.
Service IA	pytest + jeux de test fixes	Extraction des champs OCR, cohérence des prédictions, refus des questions médicales.
API	Postman / Newman	Scénarios de bout en bout : inscription → médicament → rappel → confirmation.
Front-end	Flutter test, Vitest + Testing Library	Widgets et composants clés, formulaires de validation OCR.
CI	GitHub Actions	À chaque pull request : lint (ruff, eslint, dart analyze), tests, build des images Docker.

18. Outils de collaboration

Outil	Utilisation
GitHub	Dépôt du code (monorepo : /backend, /ai-service, /web, /mobile), branches, pull requests, revues, GitHub Actions.
Jira	Product Backlog, sprint board, story points, burndown chart.
Figma	Wireframes, maquettes et prototype des interfaces.
Discord	Daily Scrum, réunions et échanges.
Postman	Tests et documentation partagée des API.
Google Drive	Documents de projet et présentations.

19. Sécurité, confidentialité et limites médicales

- Principe du moindre privilège : chaque rôle ne voit que les données nécessaires (permissions DRF par objet).
- HTTPS obligatoire ; chiffrement des champs sensibles en base ; secrets dans des variables d'environnement.
- Consentements distincts pour le partage avec un aidant ou un professionnel ; partages révocables et à durée limitée.
- Limitation du débit (throttling) sur l'authentification ; journal d'audit des actions sensibles.
- Politique de conservation et de suppression des données (droit à l'effacement RGPD).
- Aucun conseil de dosage, d'arrêt ou de modification de traitement généré par l'IA.
- Message clair indiquant que les rappels ne garantissent pas la prise effective du médicament.
- Étude des obligations réglementaires applicables avant toute mise en production réelle en santé.

20. Risques et mesures de réduction

Risque	Mesure proposée
Erreur OCR	Prétraitement OpenCV, score de confiance, vérification manuelle et confirmation avant sauvegarde.
Données insuffisantes pour le ML	Générateur de données synthétiques ; modèle présenté comme expérimental ; comparaison avec la baseline.
Notifications non reçues	Celery avec retry, notifications locales en secours sur mobile, tests des permissions et des modes d'économie d'énergie.
Hallucinations de l'assistant	RAG limité aux sources approuvées, prompt restrictif, citation des sources, refus des questions médicales.
Coût ou indisponibilité du LLM	Possibilité de basculer vers un modèle local via Ollama.
Accès non autorisé	JWT, RBAC, consentement, journalisation et tests de sécurité.
Projet trop ambitieux (Web + mobile + 3 IA)	Périmètre MVP par plateforme : mobile pour le patient, Web pour le tableau de bord ; fonctions « Could » reportées si nécessaire.

21. Conclusion et perspectives

Fakarni propose une solution numérique pour faciliter l'organisation et le suivi des traitements. Sa valeur repose sur un parcours simple — enregistrer les médicaments, planifier les prises, recevoir des rappels, confirmer et consulter l'historique — enrichi par trois modules intelligents : OCR avec extraction structurée, prédiction explicable du risque d'oubli et assistant RAG encadré. L'architecture découplée (Django, Celery, FastAPI) garantit la fiabilité des rappels et l'évolutivité des modules d'IA.

Perspectives

- Reconnaissance de l'écriture manuscrite (modèles TrOCR) pour les ordonnances.
- Accessibilité : commandes vocales, police adaptable, plusieurs langues (dont l'arabe).
- Intégrations avec des dispositifs ou services compatibles, après étude de sécurité.
- Évaluation des modèles sur des données réelles anonymisées, dans un cadre éthique approprié.

22. Questions probables du jury et réponses

Question	Réponse proposée
Qu'est-ce qui différencie Fakarni ?	L'intégration, dans un même parcours, de l'OCR avec validation, du suivi des prises, de la prédiction du risque d'oubli et d'un assistant RAG, avec un partage révocable. Les concurrents étudiés ne mettent pas ces fonctions en avant.
Où est vraiment l'IA ?	Trois modules : vision + NLP (OCR et extraction de champs), apprentissage supervisé (prédiction d'oubli suivant CRISP-DM) et IA générative (assistant RAG).
Pourquoi FastAPI en plus de Django ?	Pour isoler les dépendances lourdes de l'IA, les faire évoluer et les déployer séparément sans impacter l'API métier.
Pourquoi Celery et Redis ?	Les rappels doivent partir à l'heure même si aucun utilisateur n'appelle l'API : il faut un planificateur et une file de tâches avec relance automatique.
Comment entraîner le modèle sans données ?	Avec un générateur de données synthétiques reproduisant plusieurs profils de patients ; le modèle est comparé à une baseline et évalué par rappel, précision et F1.
Que faire si l'OCR se trompe ?	Les champs extraits restent des propositions, avec un score de confiance ; l'utilisateur doit les vérifier et les corriger avant tout enregistrement.
L'assistant peut-il donner des conseils médicaux ?	Non. Un garde-fou détecte ces questions et redirige vers un professionnel ; l'assistant ne répond qu'à partir des sources approuvées.
Le projet n'est-il pas trop ambitieux ?	Le périmètre est réparti par plateforme et par priorité MoSCoW ; le MVP (Sprints 1–2) est utilisable sans les modules d'IA, qui sont ajoutés de façon incrémentale.

Sources de l'état de l'art

- Medisafe — fiche App Store et Google Play (MediSafe Inc.) ; pillo.care, comparatif Medisafe 2026 (limitation de l'offre gratuite).
- MyTherapy — fiche App Store (smartpatient GmbH) ; mytherapyapp.com.
- CareClinic — fiche App Store (CareClinic Software Inc.) ; careclinic.io.